

Teaching Agile Collaboration Skills in the Classroom

Martin Kropp
University of Applied Sciences
Northwestern Switzerland
Windisch, Switzerland
martin.kropp@fhwn.ch

Andreas Meier
Zurich University of Applied
Sciences
Winterthur, Switzerland
meea@zhaw.ch

Robert Biddle
Carleton University
Ottawa, Ontario
Canada
robert.biddle@carleton.ca

Abstract—Agile methodologies like Scrum or Extreme Programming have come a long way over the last fifteen years. Recent quantitative studies show that many companies have successfully adopted agile methodologies. It was found that in agile software development, experience leads to collaboration. It could also be shown that successful professional agile teams tend to use more collaboration practices. In 2013, the new Computer Science studies at the University of Applied Sciences were started. For this, a new curriculum was developed. This paper presents and discusses the lectures, labs and educational software projects in the programming and software engineering modules. It is discussed how agile collaboration and collaboration practices can be taught in the classroom. For this, the setup and observations of an agile student project are presented and different online collaboration tools are discussed. It is argued that software engineering education benefits significantly from embracing the modern collaboration tools the Internet has made available.

Keywords— *Software Development, Curriculum, Collaboration, Agile.*

I. INTRODUCTION

Agile software development has become a mainstream approach in software project management as recent studies show [1], [2]. With this trend, also many less experienced non-expert teams have started applying agile approaches. The authors have conducted several studies, which show that especially collaborative practices like pair-programming, daily stand-up and retrospectives are rarely applied by less-experienced agile teams. The usage gradually evolves over time with increasing experience in agile development. The studies show that agile methodologies are of high practical relevance. Furthermore, it appears that there is a high demand for well-educated software engineers in the agile domain, and that there is a significant lack of knowledge in agile development. Another phenomenon which was found was that distributed development, either with near-shore or off-shore teams, has become standard. This makes collaboration more challenging, and makes it important to work with appropriate collaboration tools.

The studies delivered excellent input for further evolution of undergraduate and graduate software engineering education. It showed what:

- the current state of the art in practice is
- competences are needed in industry
- competences are lacked in industry

Based on the results, which are presented in section III.A, and our experience in teaching software engineering and agile methods for more than a decade, we started developing a new software engineering curriculum. It especially addresses agile methods with a much higher emphasis on teaching collaboration practices and agile values. Special attention was paid to also integrating Internet based collaboration tools, as enablers for software development in distributed teams. In this paper we present this new curriculum approach, describe their first experiences and discuss further improvements.

The rest of the paper is structured as follows: Section two discusses related work. Section three presents results from the various studies and their impact on the new curriculum. Section four presents the new curriculum, and especially also addresses agile methods and the practices that need to be taught. Section five presents how collaboration and agile values are taught and practiced in the classroom and the usage of appropriate collaboration tools. Section six evaluates the observations, followed by final conclusions and ongoing evolution of the curriculum.

II. RELATED WORK

Though it took quite a while before agile development methods were noticed by the academia, there are now many courses offered at universities all over the world that teach the agile approach.

Schroeder et al. [6] report on the setup of two software development labs specifically designed so that the social interaction – “how to deal with collaborative software development processes and their need for self-organization, motivation, and work coordination” – is taken into conscious consideration. Scharf and Koch [7] present the design of a Scrum based undergraduate software engineering course, which is based on role-playing. To ensure that their students understand “agile” thoroughly, they designed the complete course as realistically as possible. For instance, they make research assistants play customers and a student play the Scrum master. Stettina et al. [8] discuss their experiences with

teaching software engineering practices and their continuous improvement. They emphasize the importance of an intensive coaching routine based on agile practices. In [9] Maurer and Anslow report about their experiences teaching a group based agile software development project course. They report that students enjoy the interactive exercises and the self-organized teams, but also that it is challenging getting the right customer involvement. Rico and Sayani present in [10] their results

With respect to teaching agile methods, it is of interest which practices are considered to be important for successful projects and which are applied. We divided the practices into engineering practices, and management practices. The first group refers to technical practices, which are applied individually, i.e. unit testing, continuous integration, clean code, etc. The second group encompasses the more collaborative practices like pair programming, the daily

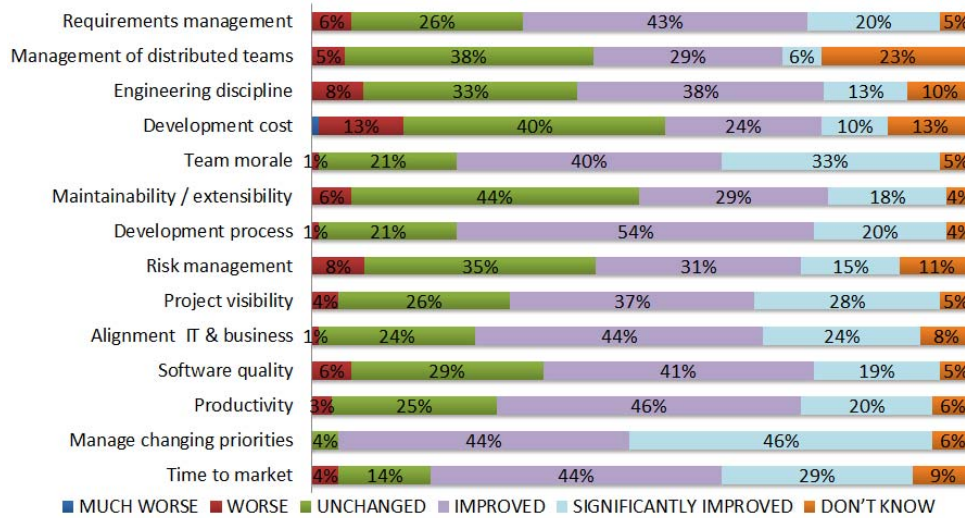


Figure 1. Influence of Agile Methods

teaching student teams agile methods in their final year course. They report that those teams finding the optimal balance of customer collaboration, technical programming and use of agile methods had a better productivity.

III. LEARNING FROM PRACTICE

A. Quantative Study

The authors have conducted a bi-annual online survey, the Swiss Agile Study (SAS), about the application of software methodologies and practices in Switzerland. The first study was conducted 2012, followed by a second one in 2014. With roughly 100 participating companies and 150 IT professionals, it is the largest independent study in Switzerland. The full study report was published in [2].

The results of the latest study show that agile software development has become the dominating approach in Swiss IT companies. More than 70% of the participants stated that they follow an agile approach [2].

Companies applying an agile approach report significant improvements in time-to-market, change management and handling changing priorities as shown in Fig.1.

standup and other agile meetings.

From the viewpoint of education it is also interesting to see which agile practices are seen to be important for project success by the practitioners. Fig. 2 and Fig. 3 show the importance of the various practices for the project success, in decreasing order.

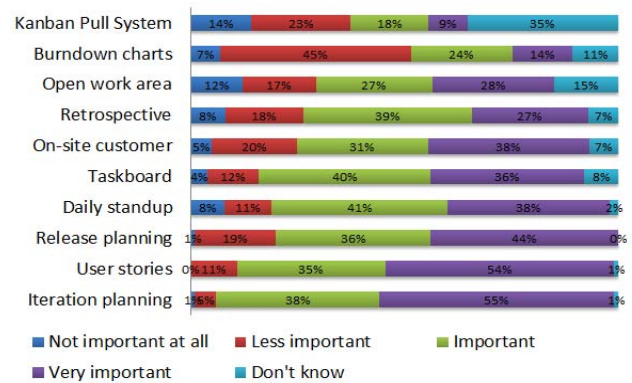


Figure 2. Importance of management practices

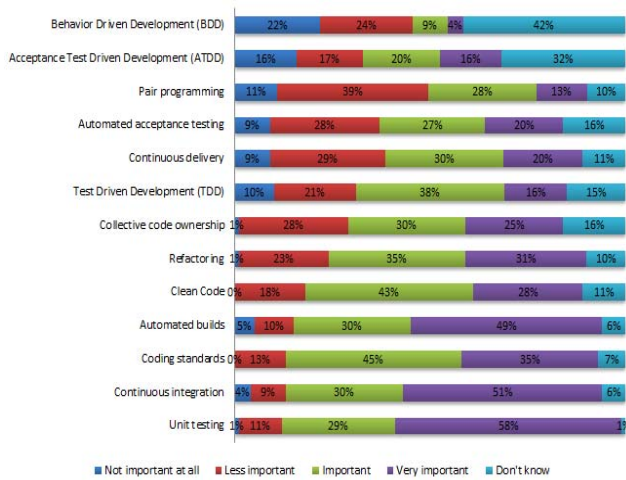


Figure 4. Importance of technical practices

In the study, we also asked which practices are actually applied in the daily work and analyzed the data with respect to how experienced the companies are in agile development. The following co-occurrence matrices in Fig. 3 show the results to the question “Which of the following practices could be observed by someone visiting your company in the next month?” with respect to the experience the companies have in agile development. In the co-occurrence matrix all practices are along the x-axes and y-axes. The diagonal shows the percentages of how many companies use the corresponding practice. Blue color means high usage values, red colors low values. The other entries show how many companies use the two practices in common. The basic technical practices are listed in the upper rows, followed by collaborative practices in the middle rows and the more advanced practices in the bottom

rows. The left matrix shows the companies with less than 3 years of agile experience, the middle matrix those with 3 to 5 years, and the right one shows those with more than 5 years of experience. The figure clearly shows that companies new to agile apply the basic engineering practices like unit testing, continuous integration and coding standards. The usage rate increases still with more experienced companies. However, collaborative practices like pair programming, daily standup, retrospectives, on site customers, or open work areas are hardly used by new agile companies and only slowly increase with more experience.

From the perspective of software engineering education this could mean that the basic practices are essential skills that they should be taught and practiced from the basic undergraduate courses on. Special emphasis should be put on the collaborative practices, which are seen as very important as shown in Fig. 2 but used only by experienced agile companies. More advanced practices like ATDD and BDD that are rarely used even by experienced companies, could be taught in upper undergraduate courses or in graduate courses.

B. Qualitative Study

We also conducted an interview study with eight agile companies in Switzerland asking them about the essential success factors in their agile projects. The complete study report is published in [3].

In the interviews, all the teams reported that they emphasize the engineering practices. These practices are seen as very important and as a kind of foundation, which ensures that software can be developed in short iterations with the required quality. There is a constant process of improvement and refining of these practices. Depending on the maturity of the team and organization, this is triggered by the agile champion (see below), the team members or, in the case of an agile

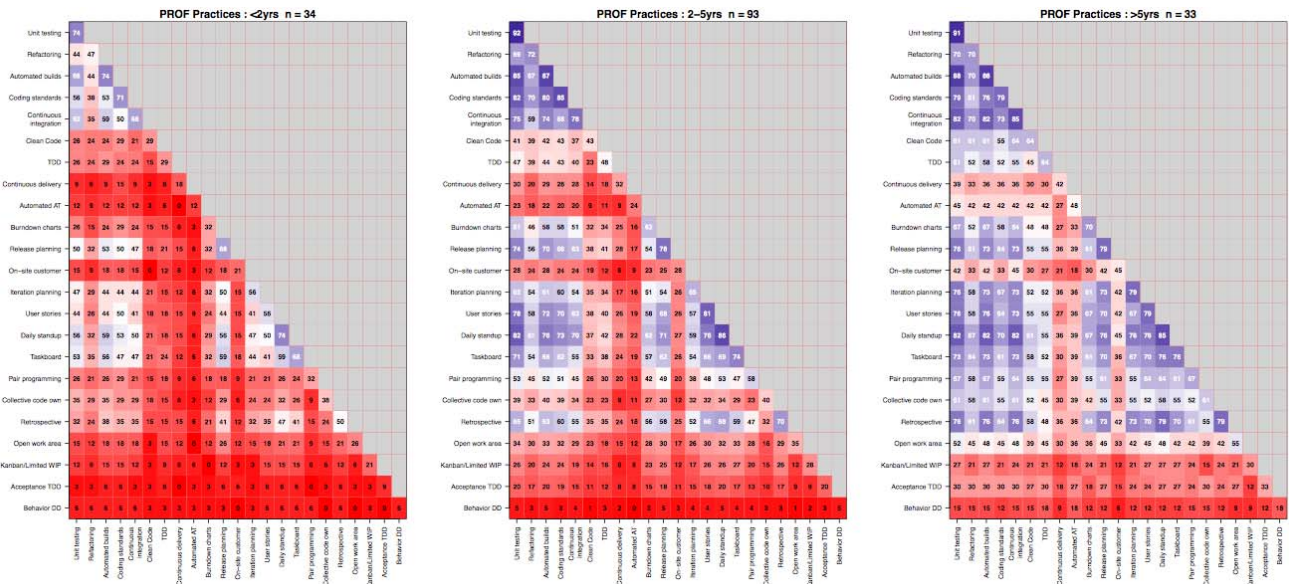


Figure 3. Applied practices by experience of the IT professionals

company, even by the management. Refactoring, automated tests, continuous integration and deployment, pair programming, test-driven development, etc. are the tools of the trade. Testing, continuous integration and clean coding was mentioned as core practices in most organizations.

We present the findings following the three levels of the Agile Competence Pyramid.

1) *Technical Practices*

Testing: In almost all the organizations automated testing on unit level is well established and is seen as an absolute must for providing a good software quality. More mature organizations also apply automated testing on acceptance testing level using new approaches like Automated Acceptance Testing (ATDD) and Behavior Driven Development (BDD).

Continuous Integration: Continuous integration is seen as an absolute must for being able to deliver software with high frequency. Thus all organizations have established an automated build and test environment, which provides immediate feedback to the developers about the quality of the system being built. Some organizations already strive for continuous delivery to automate the delivery process for faster and safer delivery to customers with less effort.

Clean Code: Continuously paying attention to writing good code from the very beginning is considered more and more important. Some of the organizations started applying the Clean Code [14] approach with the goal of establishing a constantly high code quality. These teams also apply further practices like continuous quality control, regular or even institutionalized code reviews, and regular pair programming.

2) *Collaboration Practices:*

All teams in our study originally started with Scrum [16]. However, most teams have carefully altered Scrum over time and customized it to the specific needs of their organizations and projects. The ability to alter the process is important because it allows the elimination of the impediments that hinders the team from being efficient. Retrospective is a good example of how to handle the dampening of the parts that fail and the amplifying of the parts that succeed. The interviewed companies especially emphasized the following collaboration aspects:

Customer and Requirements: In all the interviews, it was pointed out that an intensive and frequent communication with the customer was of outmost importance. In all the successful projects there was a very good understanding of the needed requirements by all team members. Communication solely with the Product owner is unsatisfying. Developers feel the need to communicate directly and get feedback from the end-users. User Stories [18] is the premier method to express functional and even non-functional requirements on cards.

Collaboration and Communication: Intensive and open communication among all stakeholders is seen to be one of the key elements for successful agile projects. In our interviews we identified three major communication scenarios. The team members themselves must communicate among each other

intensively. The team as a whole must communicate with the customer and end-users, and finally the team must establish good communication with the management (which is often organized in a classical hierarchical way).

To foster communication in teams, almost all organizations implement co-located teams in an open office environment. Most communication is then carried out informally over the desk. This helps to reduce official meeting time significantly and make the necessary meetings much more efficient.

Regular communication with the customers is established through short iterations with reviews and continuous feedback loops. However, developers have a clear need to communicate directly with end-users instead of the product owners.

Communication and collaboration with remote teams is always seen as challenging. Companies invest a lot in establishing a good communication between the teams. Means are regular chat sessions, remote participation in meetings, and video conferencing. Despite its high costs, some companies value bringing the people together physically at regular intervals. A week of common work usually “refreshes” relationships for about 5-6 weeks, and then the next physical meeting is due.

3) *Agile Values*

All interviewed organizations mentioned that developing agile is not only a matter of changing processes, but foremost, a change of the culture in an organization. Thus values become more important. Scrum defines the following five Scrum values [16] (in no particular order): Commitment, Focus, Openness, Respect and Courage. Extreme Programming [17] defines the values a bit differently: Simplicity, Communication, Feedback, Respect and Courage. In the following we present the findings concerning agile values.

Transparency and Openness: The study showed that in all projects both transparency and openness were valued very highly. Openness of the individuals also means that the team members not only take responsibility for a dedicated area of the project, e.g. requirements, components, etc., but are open and willed to understand the system as a whole, to have the big picture in mind. People in agile teams take responsibility for what they do in the context of the project as a whole.

Craftsmanship: The idea of highlighting the importance of software development practice has been popularized in [14]. The author introduces the disciplines, techniques, tools, and practices of software craftsmanship. Craftsmanship is much more than a technique: It is about attitude. The author shows how to approach software development with honor, self-respect, and pride; work well and work clean; communicate and estimate faithfully; face difficult decisions with clarity and honesty.

In our interviews, we found that members of agile teams and organizations take pride in their work. They seek mastery in their (programming) skills. They know the importance of producing high quality software and great value. In the interviews, all the teams and organizations pointed out that craftsmanship, or whatever they named it, was important and that most developers had a high motivation to become better

experts. We found that they challenge themselves and they challenge their colleagues to continuously grow and develop.

C. Agile Competence Pyramid

Developers in agile software development should possess a set of different competences. These competences can be divided into three categories as shown in the Agile Pyramid of Competences in Fig. 5. The pyramid was first presented in [21]:

- Technical Practices
- Collaboration Practices
- Agile Values

The pyramid was initially developed from an educational point of view but proved to be useful as a guideline for the kind of competences an agile developer should possess. The analogy of the pyramid visualizes the decreasing number of required



Figure 5. Agile Competence Pyramid

skills or competencies for an agile software developer from bottom to top. On the other hand, it reflects the increasing difficulty to learn or acquire these competences. Technical practices can be taught very well in the classroom or be learned by the individuals on the job at their own pace. Collaboration practices are, in our experience, best learnt through projects in teams. The most difficult competences to teach or learn are the agile values on top of the pyramid, since they often require a change in the attitude of the individual.

With the pyramid we also visualize the order in which the different practice levels should be acquired. Learning the collaboration skills before a developer has a good working knowledge of the underlying engineering skills is like trying to build a pyramid from top to bottom. It might be possible, but at a very high cost. Unfortunately, many companies are using this approach: they send their developers to a two day Scrum Master course and expect them to be agile after that. The same is valid for the agile values. Developers usually hear about the values in an Agile or Scrum course. But to properly understand and act accordingly requires a lot of experience and time.

The pyramid of agile competences served as a guideline in the development of the new curriculum. The next chapter presents the details of the curriculum.

IV. THE NEW COMPUTER SCIENCE CURRICULUM

In fall 2013, the new Computer Science studies at the University of Applied Sciences were started. For this, the curriculum [1] was updated and a lot of content newly developed. In this paper we will have a closer look at the lectures, labs and project modules in the *programming* and *software engineering* module.

The total workload of the Bachelor program is 180 ETCS¹ credits. There are different study models: a three-year full-time study, a four-year part-time study with or without a practice-integrated Bachelor. The content of the four-year course corresponds to that of a full-time academic program with 180 ECTS credits. In the first two years, programming and software development make up over a quarter of the ETCS credit. Four ETCS credits usually translate to a 90 minutes lecture and a 90 minutes programming lab a week.

TABLE I. OVERVIEW SOFTWARE ENGINEERING OF FULL-TIME CURRICULUM (UNDERGRADUATE)

	Programming in Java	Software Development	Project Module
1.Semester	4 ECTS	-	2 ECTS
2.Semester	4 ECTS	-	6 ETCS
3.Semester	-	4 ECTS	4 ECTS
4.Semester	-	4 ECTS	4 ECTS

Part-time studies are designed in such a way that working while studying is supported to the greatest extent possible. For example, the timetable is designed compactly and credit may be given for professional activities.

In the development of the new curriculum, the experience from previous lectures and research [21] [22] was integrated. The curriculum was purposely designed with both plan-driven and agile methodologies in mind.

A. First Year: Introduction to Programming with Flipped Classroom

In the first year, the main focus is on general programming concepts from a software engineering perspective². The students are taught object-oriented programming with Java. The first semester is quite extraordinary because we teach an “objects-first approach” [5] in combination with the “flipped classroom” concept [11].

¹ ETCS = European Credit Transfer System: uniform European evaluation system of performance criteria as part of the Bologna Reform. Around 30 hours of student work corresponds to one credit. Full-time students study an average of 30 ECTS credits and part-time students 22 ETCS credits per semester.

² Computer Science concepts are taught in different lectures and are not discussed in this paper.

One of the reasons for choosing “objects-first” was that it allows an approach where teachers truly deal with the important concepts first. We use Java as programming language and *BlueJ* [12]. “*BlueJ* is a Java simple integrated development environment (IDE) that is being developed and maintained at the University of Kent at Canterbury, UK, and La Trobe University, Melbourne, explicitly as an environment for teaching introductory object-oriented programming.” [12] With *BlueJ*, students can create and interact with objects directly. This has the advantage, that important object-oriented concepts such as classes, objects, methods and parameters can easily be discussed in a concrete manner before looking at the details of Java syntax. After the students become familiar with Java, they are encouraged to switch to a professional IDE like Eclipse [13].

Flipping the classroom has the benefit that students can explore topics in greater depth during class time. Solving and discussing programming exercises in groups or the whole class create meaningful learning opportunities. The teacher explicitly chooses the best-suited exercises to reinforce the understanding of the topic.

In a flipped classroom, content delivery may take a variety of forms. We use a combination of text reading and online collaboration. The course is based on the excellent textbook “Objects First with Java – A Practical Introduction using BlueJ” [5]. For online collaboration, a learning management system (LMS) is used. The textbook together with the information on the LMS³ delivers all the necessary content outside of the classroom. Online quizzes are used to assess the students’ progress and give them instant feedback. Furthermore the quizzes are important because they are the basis for choosing the above-mentioned programming exercises.

Next to the programming introduction, the students learn to write automatic tests with *JUnit*, the benefits of *clean code* [14] and how to use a code versioning system (CVS).

In the first year, there is also a project module (Table I) where the students learn to develop software with a plan-driven methodology, i.e. waterfall. In the first semester they write a requirements specification and in the second semester the software is developed, tested, reviewed by a lecturer and deployed.

B. Second Year: Focus on Software Engineering

In the second year, the focus is on software development with iterative and agile methodologies. In the third semester, the students are taught the “Unified Process” (UP) [14]. Since the authors are not involved in this part of the curriculum, UP will not be further discussed in this paper.

In the fourth semester, the focus is on agile methodologies: Scrum [16] and eXtreme Programming (XP) [17]. In the lecture the Agile Manifesto is discussed in the lecture and the students are introduced to agile software development, i.e.

Scrum. The students learn about Continuous Integration (CI) and how to configure a CI-Server. The importance of automatic testing is reinforced; test-driven development (TDD) discussed and applied. The educational objective of the last semester is to sum up the technical and collaboration practices as presented in chapter III.C

In the fourth and last semester of the basic study, the students develop a larger software application in an agile team in the second project module. Details and motivation will be presented in section V.B.

V. COLLABORATION IN THE CLASSROOM

One of the key differences between agile and plan-driven software development methodologies is the emphasis on collaboration and feedback. In Scrum, the predominant agile methodology, the collaboration practices and values are central. Therefore it is important that the students learn to apply them in a realistic classroom setup.

In the new curriculum this is done in three-steps:

1. Students learn the theory of Scrum and XP
2. Students play the *Scrum Lego City game* as first hands-on experience.
3. Students execute an agile software development project over one whole semester in “real” teams of five to nine members.

A. Scrum Lego City Game

As a hands-on introduction to Scrum, the authors usually play the Scrum Lego City game [20] with the students. Over the years, we have played this game many times. It has always been very educational, interesting and a lot of fun. The game addresses every role, ceremony and Scrum principles: Starting with the product vision then presenting the product backlog, making the sprint planning, developing the Lego city (product) and ending with the review meeting and the retrospective. The game takes between two and three hours and is played in teams of 5 to 9 members. The teacher acts as customer – or *Product Owner* in Scrum terminology – and guides the team building while building their city with Lego bricks after their product visions.

Scrum is all about teamwork, respecting people, cooperation, understanding and productivity. The Scrum Lego City Game is designed to make students understand and “feel” how and why Scrum works. The game promotes collaboration and uses as much as possible of what has been learned about Scrum in theory

B. Classroom Setup for Agile Collaboration

1) Maturity leads to Collaboration

In [23] it was found that in agile software development, experience leads to collaboration. It could also be shown that successful professional agile teams tend to use more collaboration practices. Unfortunately, this transition to more and better collaboration tends to take many years. This raises

³ The Internet simplifies flipping very much: We use a web-based e-learning platform for teaching, learning, assessment and communication. LMS can be used to provide educational content, form groups, organize users, and assign users to the courses. You can learn, assess, communicate, and share.

two questions: What are the barriers and can this transition to maturity be accelerated?

TABLE II. BARRIERS AND DIFFICULTIES

	Comp	Prof
Ability to change organizational culture	29%	37%
Availability of personnel with necessary skills	18%	14%
Customer collaboration	16%	14%
Project complexity or size	22%	12%
General resistance to change	4%	9%
Lack of management support	3%	6%
Budget constraints	5%	4%
Confidence in ability to scale agile	1%	3%

Table II gives some hints to the first question. In the Swiss Agile Study [1] the participants were asked about the top barrier in their company. (Item: “What are the current barriers and difficulties to further adopt agile software development in your company?”). As can be seen, culture, skills and customers are the top barriers

In the following paragraphs, we present answers to the second question.

2) Educational Software Development Project

In an educational software project, we are in a privileged situation because all the barriers listed in Table II are either irrelevant or can be minimized by choosing the project setup properly. Still, professional and educational projects differ in many ways and we have to be careful not to end up with mimicry.

Some of the advantages in an educational setup are:

- Students can choose a project vision together with the coach
- Project has a small scope and no “real” budget constraints
- Students can collaborate with an “ideal” customer since the coach acts as customer
- Students can apply all technical and collaboration practices
- Students can experience agile values
- Students neither have to work with legacy code nor worry about the intricacies of a production environment

Some of the impediments in an educational setup are:

- Students usually only have one project meeting per week

- Students only work sporadically on the project
- Students often work on the project at different times
- Students are not co-located, i.e. the daily stand-up meeting is not feasible
- Students are less experienced

In the classroom, it is possible to set up an environment with little or no impediments, which makes this transition much easier and faster. The reminder of this chapter describes the setup of the student projects and the experience made in respect to agile project management and collaboration practices. Table III gives the outline of the complete agile student projects.

Scrum: We chose Scrum [16] as the methodology because of the good experiences we had made in industry and research projects. Further, Scrum is the most widely used agile methodology [1] in Switzerland. For the educational development project, Scrum was adapted to fit the special requirements in the classroom.

Optimal Student Teams Size: Optimal development team size is small enough to remain agile and large enough to complete significant work within an iteration or Sprint. In Scrum the recommendation is seven, plus or minus two⁴ [16]. Fewer than five team members decrease interaction and collaboration, which might result in smaller productivity. Smaller teams might also encounter skill constraints. More than nine members require too much coordination. Large development teams generate too much complexity for an empirical process, like Scrum, to manage.

What about the optimal size of student teams? The authors had best results when the team size was at the upper limit. In smaller student teams there is a risk that the students do not realize the necessity of proper project management. Much too often, project management is seen as an overhead and, in the worst case, only applied because the coach says so. In properly sized teams, project management is crucial for success. Another benefit of larger teams is, that the students get more work done, which results in more realistic software applications. In our experience optimal student team size is between seven and nine members.

Automatic Testing: Next to team size, we demand that the teams do rigorous automatic testing. Automatic testing is difficult, but in combination with refactoring it has a profound effect on the code quality. It is important that the students have a test concept and test environment ready before they actually start coding! Therefore we introduced a “Sprint Zero” before the official project start. The goal of Sprint Zero is simple: All teams are able to do automatic testing.

The students have to implement a complete test case, which demonstrates their ability to test the application from end-to-end. At the end of every regular Sprint, code coverage is inspected and discussed with the students.

⁴ The roles of Product Owner and Scrum Master are not included in this count unless they are also completing the work of the Sprint Backlog.

TABLE III. OUTLINE OF AGILE STUDENT PROJECTS

Before start of semester	Students and faculty are informed about the project module and asked to send in project ideas.
Week 1 (Kick-off meeting)	Kick-off. Presentation and election of best-suited project ideas. Composition of student teams
Week 2 (Sprint 0)	Sprint 0: Students setup complete automatic testing and building environment- Introduction to IT-infrastructure
Week 3	Sprint 0: Students setup complete automatic testing and building environment
Week 4 (Project start: Sprint 1)	Demonstration of testing and building environment Sprint and Release Planning
Week 6 – 14 (Sprint 1 - 5) Every even week	Sprint Review & Sprint Retrospective Planning meeting of new Sprint Development of application
Week 16	Final presentation of application End of project

Continuous Integration (CI): Continuous Integration is seen as an absolute must by many professionals to be able to deliver software with high frequency. For the student projects, the concept of automatic builds and CI are introduced. Every team uses Maven [24] and Jenkins [25] to build and integrate their software.

Clean Code and Craftsmanship: The idea of highlighting the importance of software development practice was popularized in Clean Code: A Handbook of Agile Software Craftsmanship [14]. The author introduces the disciplines, techniques, tools, and practices of software craftsmanship. Craftsmanship is much more than a technique: It is about attitude. Starting in the first semester, the students are advised to follow the Clean Code guidelines. The authors have observed that following and discussing these guidelines has a profound impact on the quality and design of the code base.

Sprint Meetings: At the end of every Sprint, the coach attends both the Sprint Review and Sprint Retrospective meetings of every student team. In these meetings, the coach learns whether the project is on track or not.

In the beginning of the new Sprint, the coach attends the Sprint planning meeting. In this meeting, the coach acts as the Product Owner. As Product Owner, the coach can steer the project in the proper direction.

All three meetings usually take place on the same day. The Sprint Review takes about 30 minutes, the Sprint retrospective about 15 minutes and the Sprint planning, depending whether User Stories have to be estimated or not, between 15 and 45 minutes. These meetings are an excellent occasion to get a good overview of all projects and therefore the coach should attend all of them.

Final Presentation: At the end of the project module, all the teams present their application in the plenum. This is a great opportunity for all teams to present their achievements. The

presentation takes about 20 minutes and consists of the project vision, architecture and design of the application as well as a demonstration.

C. Internet simplifies Collaboration

In the educational student project, the team is mostly not co-located. Additionally, the students often work at different times. This makes collaboration difficult because face-to-face communication is not easily possible. Furthermore, the teams cannot use physical task boards either. Fortunately, there are online task boards, which can be a good substitute for physical ones.

In this chapter we will present some collaboration tools, which help to circumvent the impediments in “distributed” student teams

An agile project rapidly and reliably produces new know-how and new knowledge. Good collaboration tools make the exchange of new know-how and new knowledge between the single team members and the coach possible. Without the Internet and its online collaboration tools, distributed student teams would not be feasible. The tools simplify collaboration among students and make progress visible for the coach. The same is true for problems and difficulties.

1) Jenkins CI-Server

Jenkins [25] is an open source continuous integration server. It provides immediate feedback of the build status and the overall project progress, both to the students and the coach.

2) GIT and GitHub

Git [26] is a distributed code versioning system (CVS) designed to handle everything from small to very large projects with speed and efficiency. Its workflow make it particularly well suited for distributed software development teams. Git is the successor of SVN (subversion) and is rapidly becoming the most widely used CVS.

GitHub [27] is a web-based Git repository, which is often used for hosting open-source software projects. GitHub provides several collaboration features such as bug tracking, feature requests, task management and wikis for every project.

3) Trello

Trello is a very simple collaboration tool for all kind of projects. Its main advantage is its simplicity. Trello runs in the web browser and almost on any mobile device. It consists of a task board that is very similar to a physical Scrum board. Since it is a web application, no software installation is needed.

In the student project, Trello is used to manage the user stories and the associated tasks. Trello is a good, lightweight alternative to professional agile tools like JIRA [29]. By looking at Trello, the coach immediately knows what is going on in the different projects.

4) *OpenOLAT Learning Management System*

OpenOLAT [30] is a web-based e-learning platform for teaching, learning, assessment and collaboration. OpenOLAT is simple and intuitive to use.

5) *Project Wiki*

In the student projects, a project wiki is used for documentation purposes. GitHub comes with a project wiki. Therefore GitHub is usually used for documentation.

6) *E-mail*

E-Mail is primarily used as means of communication between the coach and the teams. As coaches we prefer written, asynchronous communication to synchronous, like the good old phone or Skype.

7) *Skype*

Members of the student teams often use Skype to ask each other questions and communicate. The coach does not use Skype with the students.

VI. EVALUATION

A systematic evaluation of the new curriculum with respect to the learning targets in software engineering has not yet been executed for the first round. This evaluation will be carried out after the second or third round. Instead we present our first experiences and feedback from the students. In the round we a total of 45 students, and have setup five teams each with nine students.

In the first year we observed an improvement of the programming skills and code quality. The object first approach with BlueJ [5], [12] has clearly improved the understanding and depth of important OO-concepts skills of most students. Paying special attention to technical practices like automatic unit tests and clean code has had a positive effect on design, quality and documentation.

Applying a Learning Management System (LMS) was helpful for collaboration and presentation of material. This was especially true in the flipped classroom environment, because solving and evaluating the quizzes was supported by LMS. On the other side, the flipped class room concept showed to be pretty laborious in preparation.

In the second year, the highlight was unquestionably the agile project module for both students and coach. The developed application exceeded the expectations in scope, quality and student commitment. We observed that the coach's experience in agile software development is important for the outcome.

The coordination of the software development lecture and the project module was reported to be excellent. The students liked the combination of theory and practice. Next some written statements⁵ of the software development lecture and

⁵ The statements are translated from German. The statements are taken from the official evaluation system. For the new Computer Science studies, all new lectures and modules were evaluated systematically.

project module in the fourth semester. The students were asked what they liked best: "It's great that we do Agile..", "Practical relevance of the topics presented", "Good coordination of lecture and project module", "Best project module so far..", "I liked that we could choose the projects ourselves..", "Scrum!!".

From part-time students we often get feedback that they can benefit of the agile practices directly in their job.

Scrum in combination with the online collaboration tools made it possible, to keep the overview of the student projects. Coaching the projects to desired outcome and assessing the team members was possible in an efficient and transparent manner projects.

With respect to evaluation and assessment of the individual students, it is very hard to find out, who did what and how much in the project. The team setup is critical with respect to finding a well-balanced team with the necessary level of knowhow, motivation and commitment. Another critical issue are the Product Owner and Scrum Master roles. We let the students themselves overtake these roles. But they usually lack the necessary experience to fulfill the requirements to these roles.

VII. CONCLUSION AND FURTHER WORK

Collaboration between team members, customers, users and stakeholders is a very important part of agile software development. Maturity leads to collaboration; i.e. mature agile teams apply more collaboration practices. Therefore, it is important that the students learn about collaboration practices and how to live them. This is best realized in a "real" agile project.

This paper discusses how educational projects can be setup in such a way, that students can experience the importance of collaboration. The Internet and its plethora of online tools act as an enabler of collaboration in agile educational software projects. The benefits of online collaboration tools make it possible for the lecturer to coach and steer student software development projects and, last but not least, assess the students. The authors are convinced, that software engineering education benefits significantly by embracing the modern collaboration tools the Internet has made available.

REFERENCES

- [1] Version One. State of Agile Development Survey results. <http://info.versionone.com/state-of-agile-development-survey-ninth.html>, 22.10.2015
- [2] M. Kropp, A. Meier, Swiss Agile Study 2014, ISSN: 2296-2476, FHNW and ZHAW, Brugg, Zurich www.swissagilestudy.ch
- [3] M. Kropp, A. Meier, Agile Success Factors, ISSN: 2296-2476, FHNW and ZHAW, Brugg, Zurich www.swissagilestudy.ch (2015)
- [4] Curriculum Bachelor's degree Computer Science <https://www.zhaw.ch/en/engineering/study/bachelors-degree-programmes/bachelors-degree-computer-science/>, retrieved 20.10.15
- [5] David J. Barnes & Michael Kölling, Objects First with Java- A Practical Introduction using BlueJ, Fifth edition, Prentice Hall / Pearson Education, 2012 US: ISBN 978-013-249266-9 UK: ISBN 978-013-283554-1
- [6] Andreas Schroeder, Annabelle Klarl, Philip Mayer, Christian Kroiß. Teaching agile software development through lab courses. Global Engineering Education Conference (EDUCON), 2012 IEEE.

- [7] Andreas Scharf, Andreas Koch, Scrum in a Software Engineering Course: An In-Depth Praxis Report, CSEE&T 2013, San Francisco, USA.
- [8] Christoph Johann Stettina, Zhao Zhou, Thomas Bäck, Bernhard Katzy, Academic Education of Software Engineering Practices: Towards Planning and Improving Capstone Courses Based upon Intensive Coaching and Team Routines, CSEE&T 2013, San Francisco, USA.
- [9] Frank Maurer, Craig Anslow. An Experience Report at Teaching a Group Based Agile Software Development Project Course. Proceedings of the 46th ACM Technical Symposium on Computer Science Education. SIGCSE '15. Kansas City, USA.
- [10] D. Rico and H. Sayani. Use of agile methods in software engineering education. In *Proceedings of the International Conference on Agile*, pages 172-179, 2009.
- [11] Flipped classroom, https://en.wikipedia.org/wiki/Flipped_classroom, 14.10.2015
- [12] BlueJ, University of Kent, <http://www.bluej.org>, retrieved 20.10.15
- [13] Eclipse IDE, <https://www.eclipse.org>, retrieved 20.10.15
- [14] Robert C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship, 2009, ISBN 0-13-235088-2
- [15] Unified Process, https://en.wikipedia.org/wiki/Unified_Process, retrieved 20.10.15
- [16] Ken Schwaber, Mike Beedle. Agile Software Development with Scrum, 2001, ISBN 0-13-207489-3
- [17] Kent Beck, Extreme Programming Explained: Embrace Change. Addison-Wesley, 2004 ISBN 0-321-27865-8
- [18] Mike Cohn, User Stories Applied, For Agile Software Development, 2004, ISBN 0-321-20568-5
- [19] Agile Manifesto. <http://agilemanifesto.org/>, retrieved 20.10.15
- [20] Scrum Lego City Game. Agile 42. <http://www.agile42.com/en/training/scrum-lego-city/>, retrieved 20.10.15
- [21] M. Kropp, A. Meier. Teaching Agile Software Development at University Level: Values, Management, and Craftsmanship. Proceedings 2013 IEEE, 26th Conference on Software Engineering Education and Training (CSEE&T 2013), San Francisco.
- [22] M. Kropp, A. Meier, M. Mateescu, C. Zahn, Teaching and learning agile collaboration. Proceedings 2014 IEEE, 27th Conference on Software Engineering Education and Training - (CSEE&T 2014), Klagenfurt, Austria.
- [23] R. Biddle; M. Kropp; A. Meier. Agile Adolescence to Maturity: Experience leads to Collaboration (Workshop). OOP Conference 2016, Munich, Germany, Feb. 2016 (accepted)
- [24] Apache Maven Project, <https://maven.apache.org>, retrieved 20.10.15
- [25] Jenkins - open source continuous integration server, <https://jenkins-ci.org>, retrieved 20.10.15
- [26] Git - distributed version control system, <https://git-scm.com>, retrieved 20.10.15
- [27] GitHub - collaboration, code review, and code management <https://github.com>, retrieved 20.10.15
- [28] Trello - web-based project management application, <https://trello.com>, retrieved 20.10.15
- [29] JIRA - issue tracking and project management, <https://de.atlassian.com/software/jira>, retrieved 20.10.15
- [30] OpenOLAT – a learning management system, <http://www.openolat.com>, retrieved 20.10.15